VINUNIVERSITY

Lecture 4: Object-Oriented Design in Java II

Program Structure, Organization, Modular Programming, Class, Objects, and Exceptions.

Hieu Pham | Spring 2025

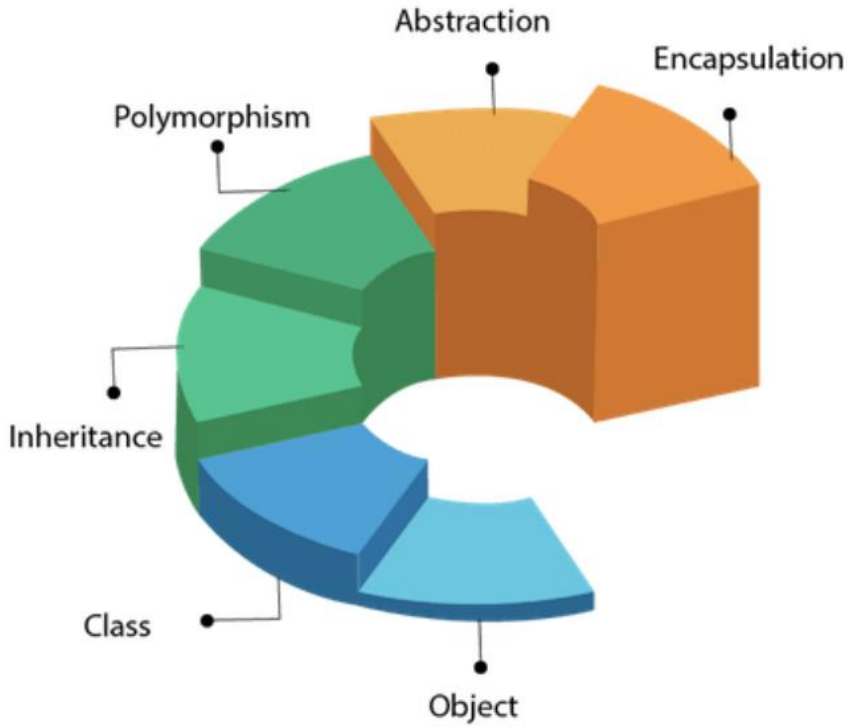
QUIZ 02

PASSCODE: FEB2725

LECTURE 03 RECAP

In lecture 03, we have investigated:

(1) The object-oriented design principles, including inheritance, polymorphism, encapsulation and abstraction.

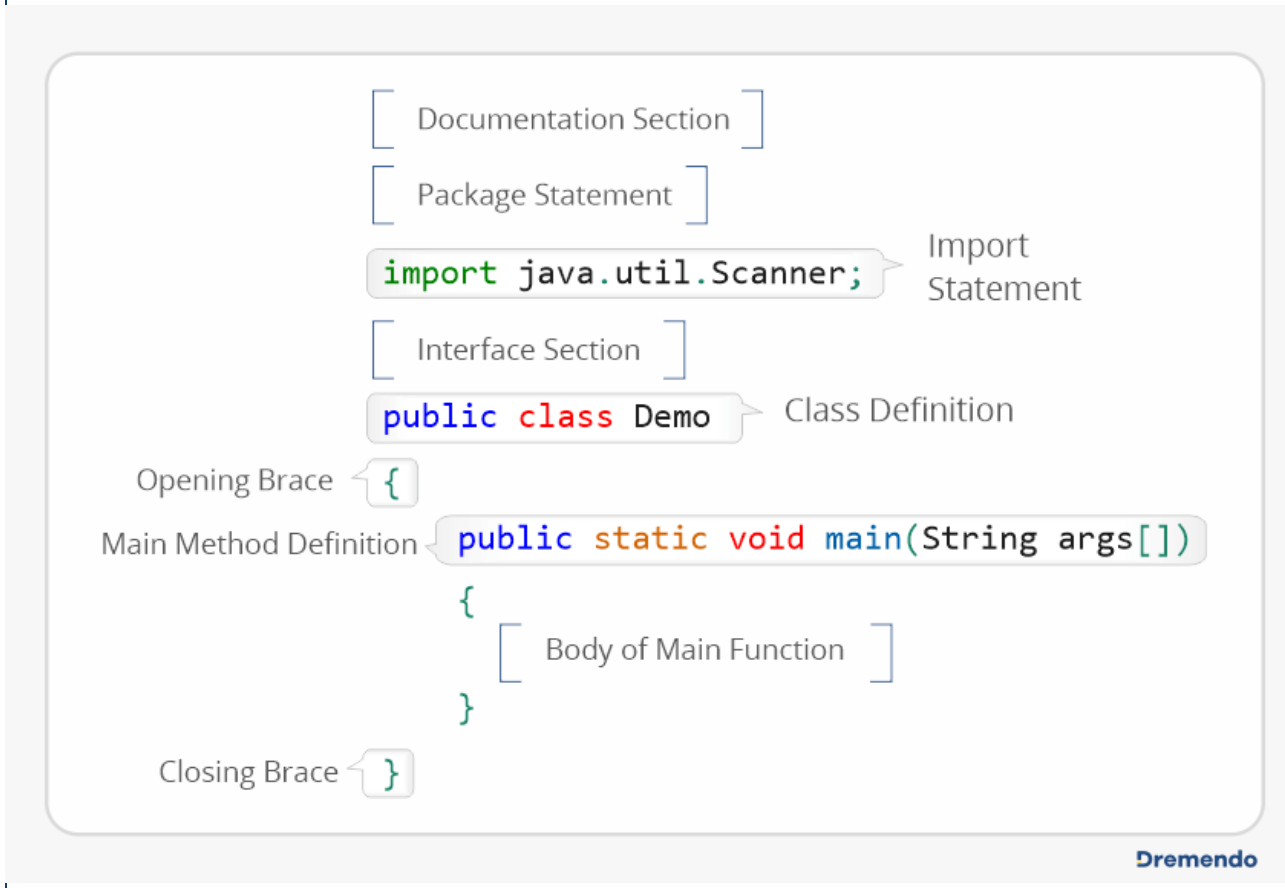


LECTURE 03 RECAP

In lecture 03, we have investigated:

(1) The object-oriented design principles, including inheritance, polymorphism, encapsulation and abstraction.

(2) Java program structure and organization



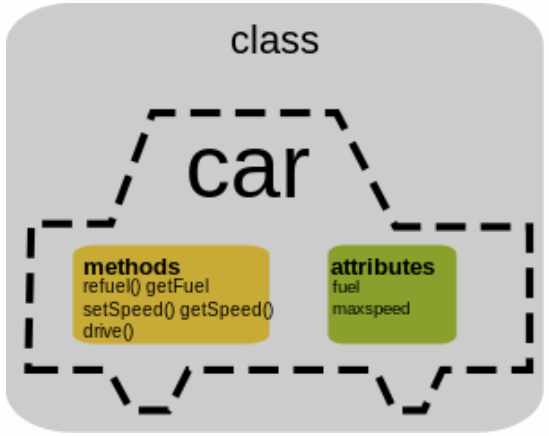
PLAN FOR TODAY

Today's lecture will cover:

- Introduction to variables in Java (i.e., class variables, instance variables, and local variables).
- Abstract classes and methods (How does Java implement data abstraction?).
- Java exceptions and catching exceptions (How does Java handle runtime errors gracefully?)
- Modular programming (How to improve maintainability, readability and scalability of your Java programs?)

JAVA CLASS *VS.* OBJECT

- A class is a logical abstraction: a physical representation of a class does not exist in memory until an object of that class has been created.
- The methods and variables that constitute a class are called members of the class.



[source]

[source]

INTRODUCTION TO VARIABLES IN JAVA

- **What are variable?** Containers that store data values.
- **Three types of variables in Java:**

1. Class variables (or static variables)
2. Instance variables
3. Local variables

CLASS VARIABLES

- Class variables (also known as static variables/fields): declared with the `static` keyword in a class, but outside a method, constructor or a block.
- They are associated with the class instead of any object.
- Class variables are shared among all instances of that class, i.e., changes made to class variables by one object will reflect in all objects.

```
public class Car {
    static int totalCars = 0; // Class variable (shared by all objects)
    String brand;

    public Car(String brand) {
        this.brand = brand;
        totalCars++; // Increments every time an object is created
    }
}
```

INSTANCE VARIABLES

- Defined inside a class but outside methods, shared by all class instances.
- Unique to each instance of the class. Across different objects, these variables can have different values.
- Does not need `static`

```
public class Car {
    String brand; // Instance variable
    int speed;

    public Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }
}
```

LOCAL VARIABLES

- Declared inside a method, constructor, or block. Cannot be accessed outside its block.
- Scope is limited to that method/constructor, or block
- Created when the method starts and destroyed when it ends.
- Must be initialized before use.

```
public class Car {
    public void drive() {
        int distance = 100; // Local variable
        System.out.println("Driving " + distance + " km.");
    }
}
```

ALL-IN-ONE EXAMPLE

1. Class variable: `classVar`
2. Instance variable: `instanceVar`
3. Local variable: `localVar`

```java
class Example {
    static int classVar = 100; // Class variable
    int instanceVar; // Instance variable

    Example(int value) {
        this.instanceVar = value;
    }

    void display() {
        int localVar = 10; // Local variable
        System.out.println("ClassVar: " + classVar + ",
            InstanceVar: " + instanceVar + ", LocalVar: " +
            localVar);
    }

    public static void main(String[] args) {
        Example obj1 = new Example(50);
        obj1.display();
        classVar = 200;
        new Example(75).display();
    }
}

# ClassVar: 100, InstanceVar: 50, LocalVar: 10
# ClassVar: 200, InstanceVar: 75, LocalVar: 10
```

QUIZZES/CLASS PRACTICE

Which type of variable would you use for:

1. A counter shared among all instances?

QUIZZES/CLASS PRACTICE

Which type of variable would you use for:

1. A counter shared among all instances? Answer: Class variable (static)

QUIZZES/CLASS PRACTICE

Which type of variable would you use for:

1. A counter shared among all instances? Answer: Class variable (static)

2. A unique identifier for each object?

QUIZZES/CLASS PRACTICE

Which type of variable would you use for:

1. A counter shared among all instances? Answer: Class variable (static)

2. A unique identifier for each object? Answer: Instance variable

QUIZZES/CLASS PRACTICE

Which type of variable would you use for:

1. A counter shared among all instances? Answer: Class variable (static)
2. A unique identifier for each object? Answer: Instance variable
3. A temporary calculation inside a method?

QUIZZES/CLASS PRACTICE

Which type of variable would you use for:

1. A counter shared among all instances? Answer: Class variable (static)
2. A unique identifier for each object? Answer: Instance variable
3. A temporary calculation inside a method? Answer: Local variable

QUESTION

Static Variable vs.
Instance Variable

What is the difference between a class/static variable and an instance variable?

CONSTANTS IN JAVA

- The constant is used in Java when a static value or a permanent value (*never changes from the initial value*) for a variable has to be implemented.
- To create a constant variable, we need to use "`static`" and "`final`" modifiers.

➢ static modifier: allows the variable to be available without loading any instance of the class in which it is defined.

➢ final modifier: makes the variable unchangeable.

Syntax for Java Constants

```
static final datatype identifier_name = constant;
```

Example

```
static final float PI = 3.14f;
```

JAVA CONSTANT EXAMPLE

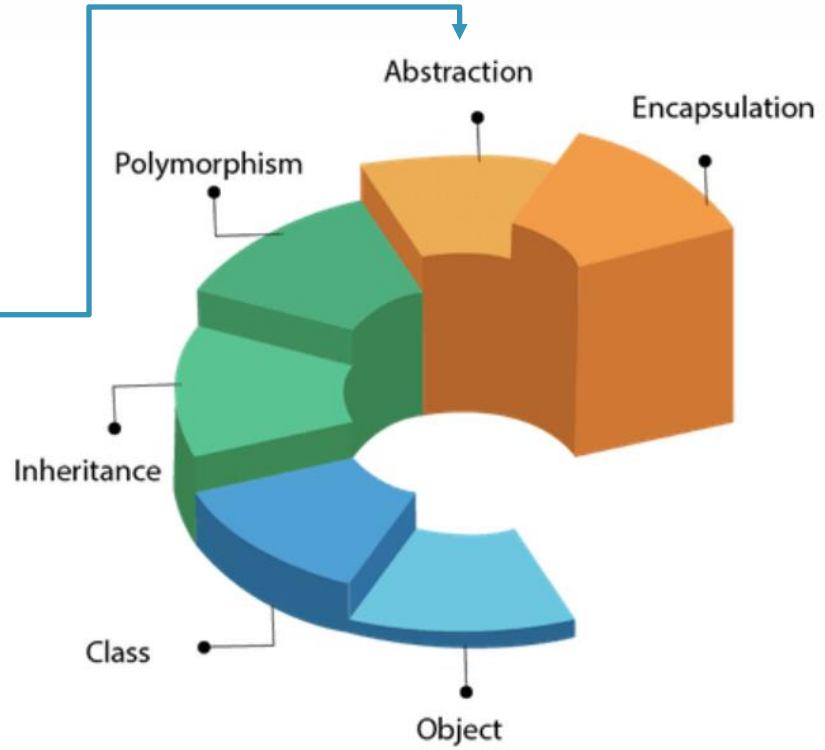
static final or final static?

```
public class Employee {

   // salary  variable is a private static variable
   private static double salary;

   // DEPARTMENT is a constant
   public static final String DEPARTMENT = "Development ";

   public static void main(String args[]) {
      salary = 1000;
      System.out.println(DEPARTMENT + "average salary:" + salary);
   }
}
```

[source]

ABSTRACT CLASSES AND METHODS

Data abstraction is the process of hiding certain details and showing only essential information to the user. Abstraction can be achieved with abstract classes



What is an Abstract class?

- An abstract class in Java is a class that cannot be instantiated (i.e., you cannot create an object of it). It is designed to be extended by other classes
- It can contain abstract methods (methods without implementation) as well as concrete methods (methods with implementation).

ABSTRACT CLASS: KEY POINTS

- Defined using `abstract` keyword.
- If a class is declared abstract, it cannot be instantiated.
- To use an abstract class, you need to inherit it from another class, provide implementations to the abstract methods in it.

ABSTRACT METHODS

- An abstract method is a method declared without an implementation (without a method body). It must be implemented in a subclass.

```
public abstract class Employee {
   private String name;
   private String address;
   private int number;

   public abstract double computePay();
   // Remainder of class definition
}
```

Abstract class allows regular methods too!

```
public class Salary extends Employee {
   private double salary;   // Annual salary

   public double computePay() {
      System.out.println("Computing salary pay for " + getName());
      return salary/52;
   }
   // Remainder of class definition
}
```

Further Reading: here

JAVA EXCEPTIONS

JAVA EXCEPTIONS

- An exception is a problem that arises during the execution of a program.
- An exception can disrupt the normal flow of the program and causes the termination of the program/application abnormally.
- Common scenarios where an exception occurs:
  1. Invalid data entered by the user.
  2. Cannot open a file.
  3. Lost network connection.
  4. JVM run out of memory.

TYPES OF EXCEPTIONS IN JAVA

- Checked exceptions (compile time exceptions): checks by the compiler, e.g., network connection errors, missing files, etc.
- Unchecked exceptions (runtime exceptions): occurs at the time of execution, e.g., programming bugs, such as logic errors or improper use of an API.
- Errors: problems that arise beyond the control of the user or the programmer, e.g., stack overflow, JVM is out of memory, etc.

CATCHING EXCEPTIONS

- Keywords: `try` and `catch`
- A try/catch block is placed within the code that is suspected to generate an exception.
- Syntax:

```
try {
    // Protected code
} catch (ExceptionName e) {
    // Catch block
}
```

Declare the type of exception to catch!

Built-in or user-defined exceptions

JAVA BUILT-IN EXCEPTIONS

Some of the important built-in exceptions in Java:

- ArithmeticException
- ArrayIndexOutOfBoundsException
- ClassNotFoundException
- IOException
- InterruptedException
- NoSuchFieldException
- NoSuchMethodException
- NullPointerException
- NumberFormatException
- RuntimeException
- StringIndexOutOfBoundsException

EXCEPTION EXAMPLE 1

```
public class MyExceptions {

    public static void main(String[] args) {

        try {
            int x = 20, y = 0;
            int res = x / y;
            System.out.println("Result = " + res);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        }
    }

}
```

EXCEPTION EXAMPLE 2

```
public class MyExceptions {

    public static void main(String[] args) {

        try {
            String myStr = "Ali Baba is Back!";
            char c = myStr.charAt(20);
            System.out.println(c);
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("String Index is Out of Bounds!");
        }
    }

}
```

EXCEPTION EXAMPLE 3

```
public class MyExceptions {

    public static void main(String[] args) {

        try {
            int a[] = new int[3];
            a[3] = 4;
            System.out.println(a[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index is Out Of Bounds!");
        } finally { //
            System.out.println("The try-catch is completed!");
        }
    }
}
```

contains all the crucial statements that must be executed whether exception occurs or not.

```
Array Index is Out Of Bounds!
The try-catch is completed!
```

MODULAR PROGRAMMING

- Modular Programming: divide a large block of code into small and independent modules which have some specific functionality. This practice enhances code readability, reusability and maintainability.
- **Advantage**:
  1. Decreases complexity in designing and writing code.
  2. Decreases duplication of code and allows reusability.
  3. Improves collaboration where each developer can work on separate modules.
  4. Easier for testing (modules can be tested independently).

MODULAR PROGRAMMING IN PRACTICE

- Java introduced modules in version 9 through the Java Platform Module System (JPMS), which allows for modularizing applications and libraries.
- Prior to that, modular programming was typically achieved through packages and classes.

Basic Example of Modular Programming in Java:
User and Product Management System

In this example, we create a modular Java application with two modules:

1. `user.management` - manages users.
2. `product.management` - manages products.

A **main application** will use both modules.

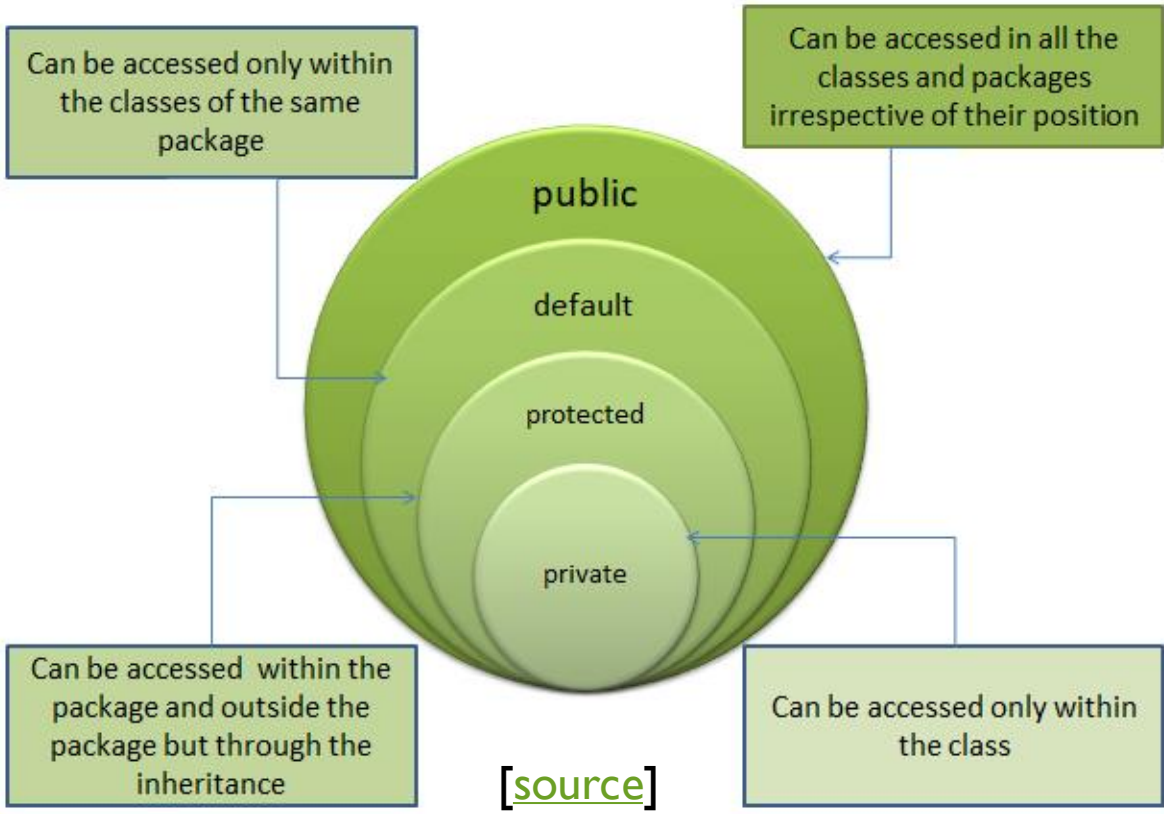
```
modular-app/
├── user.management/
│   ├── module-info.java
│   ├── com/user/management/User.java
├── product.management/
│   ├── module-info.java
│   ├── com/product/management/Product.java
├── app.main/
│   ├── module-info.java
│   ├── com/app/main/MainApp.java
```

JAVA ACCESS MODIFIERS

To specify what other classes can use this class.

```
access-modifier class ClassName {
    // some code here
}
```

Modifier	Within Same Class	Within Same Package	Subclasses in the Same Packages	Subclasses Outside the package	Outside the package (Global)
Public	√	√	√	√	√
Protected	√	√	√	√ (only to derived class)	×
Default	√	√	√	×	×
Private	√	×	×	×	×



[source]

```java
class AccessModifiers {

    // this instance variable is visible for any child class.
    public String name;

    // salary variable is visible in AccessModifiers class only.
    private double salary;

    // The name variable is assigned in the constructor.
    public AccessModifiers(String empName) {
        name = empName;
    }

    // a protected setSalary method.
    protected void setSalary(double empSal) {
        salary = empSal;
    }

    // This method prints the employee details.
    public void printEmp() {
        System.out.println("name  : " + name);
        System.out.println("salary :" + salary);
    }

    public static void main(String args[]) {
        AccessModifiers myEmp = new AccessModifiers("Ali Baba");
        myEmp.setSalary(2000);
        myEmp.printEmp();
    }
}
```

PROTECTED: EXAMPLE

```
public class Animal {
    protected String name;

    protected void setName(String name) {
        this.name = name;
    }
}

public class Dog extends Animal {
    public void bark() {
        System.out.println(name + " barks!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.setName("Max");
        dog.bark();
    }
}
```

Max barks!

PRIVATE: EXAMPLE

```
public class BankAccount {
    private double balance;

    public BankAccount() {
        balance = 0.0;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds!");
        }
    }

    public void displayBalance() {
        System.out.println("Balance: $" + balance);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(1000.0);
        account.displayBalance();
        account.balance = 500.0;
    }
}
```

Error: The field BankAccount.balance is not visible

PRIVATE: EXAMPLE

```
public class BankAccount {
    private double balance;

    public BankAccount() {
        balance = 0.0;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds!");
        }
    }

    public void displayBalance() {
        System.out.println("Balance: $" + balance);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(1000.0);
        account.displayBalance();
        account.balance = 500.0;
    }
}
```

What will be printed when running this program?

DEFAULT (INTERFACE): EXAMPLE

```
interface MyInterface {
    default void myMethod() {
        System.out.println("This is a default method.");
    }
}

class MyClass implements MyInterface {
}

public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.myMethod();
    }
}
```

Do not need to implement `myMethod()`, because it has a default implementation in the interface

```
This is a default method.
```

THANK YOU!